Docker Swarm Handson

Docker Swarm is Native clustering and orchestration tool in Docker. It automates container deployment across multiple Docker hosts. Swarm is basically a cluster of Docker Engines/worker-nodes in Swarm mode. **Manager nodes** manage state using **Raft consensus**. **Worker nodes** run the actual tasks (containers). Its basically a beginner level version of Kubernetes which has far more features (but more complex).

- Manager node: Orchestrates, makes scheduling decisions, handles Raft consensus. (master nodes)
- Worker node: Runs tasks assigned by the manager nodes.
- Service: Defines the container spec and how many replicas.
- Task: A container instance part of a service.
- Overlay network: Enables service-to-service communication ; like between app<->db.

1. Initiate Docker swarm : `docker swarm init`
   It will initiate a docker swarm cluster, with it being the only manager(as well as only worker in the swarm cluster). It will provide a docker swarm join command which can be run on other nodes (if available) so that those node join as a worker.

```
(base) labuser@ip-172-31-19-242:~$ docker swarm init
Swarm initialized: current node (vy863g1hlcaq322e5v3n0gn1p) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-501fuvfx07xcc2u19qdt2qtiif7304bhvzmwclp52qxz3clfxz-9ipibx6uugnw0q2eww94qsqu8 172.31.19.242:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

(base) labuser@ip-172-31-19-242:~$
```

2. List available nodes in swarm cluster : `docker node ls` # Note the node ID

```
(base) labuser@ip-172-31-19-242:~$ docker node ls

ID                            HOSTNAME           STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
vy863g1hlcaq322e5v3n0gn1p *   ip-172-31-19-242   Ready     Active         Leader           27.1.1
```

3. Inspect any node using Node-ID to show detailed information # `docker inspect node <<NODE-ID>>`

4. In docker swarm we deploy application as a service and we can have multiple replicas of the same service created or scaled just with the help of one command rather than doing it manually.

To deploy nginx service (Application) with 2 replicas on port 80 mapping to host port 8081 :

`docker service create --name nginx-app --replicas 2 -p 8081:80 nginx:latest` # --replicas=2 starts 2 containers (tasks).

```
(base) labuser@ip-172-31-19-242:~$ docker service create --name nginx-app --replicas 2 -p 8081:80 nginx:latest
fg4q63vk8zm60dzjrdygoe85v
overall progress: 2 out of 2 tasks
1/2: running   [==================================================>]
2/2: running   [==================================================>]
verify: Service fg4q63vk8zm60dzjrdygoe85v converged
```

4. Check the service/containers for nginx created on docker swarm : `docker service ls`

```
(base) labuser@ip-172-31-19-242:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS     NAMES
accf02103adc   nginx:latest   "/docker-entrypoint.â€¦"   34 seconds ago   Up 32 seconds   80/tcp    nginx-app.1.ulyt2tokfc5rudxcioq1b57bl
4f6ea528c7b9   nginx:latest   "/docker-entrypoint.â€¦"   34 seconds ago   Up 32 seconds   80/tcp    nginx-app.2.1v41lurx76mh3o1jdcyoe9067
(base) labuser@ip-172-31-19-242:~$ docker service ls
ID             NAME        MODE         REPLICAS   IMAGE          PORTS
fg4q63vk8zm6   nginx-app   replicated   2/2        nginx:latest   *:8081->80/tcp
(base) labuser@ip-172-31-19-242:~$ docker service ps nginx-app
ID             NAME          IMAGE          NODE                DESIRED STATE   CURRENT STATE            ERROR     PORTS
ulyt2tokfc5r   nginx-app.1   nginx:latest   ip-172-31-19-242    Running         Running 2 minutes ago
1v41lurx76mh   nginx-app.2   nginx:latest   ip-172-31-19-242    Running         Running 2 minutes ago
(base) labuser@ip-172-31-19-242:~$
(base) labuser@ip-172-31-19-242:~$ docker container ls
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS     NAMES
accf02103adc   nginx:latest   "/docker-entrypoint.â€¦"   2 minutes ago   Up 2 minutes   80/tcp    nginx-app.1.ulyt2tokfc5rudxcioq1b57bl
4f6ea528c7b9   nginx:latest   "/docker-entrypoint.â€¦"   2 minutes ago   Up 2 minutes   80/tcp    nginx-app.2.1v41lurx76mh3o1jdcyoe9067
```

Tasks having Task IDs, not Container ID

5. We can scale up/down the no. of task (containers) for our nginx-app task(application) using : `docker service scale nginx-app=5`

Similarly we can scale down using : `docker service scale nginx-app=1`

```
(base) labuser@ip-172-31-19-242:~$ docker service scale nginx-app=5

nginx-app scaled to 5
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service nginx-app converged
(base) labuser@ip-172-31-19-242:~$ docker service ps nginx-app
ID             NAME          IMAGE          NODE                DESIRED STATE   CURRENT STATE            ERROR     PORTS
ulyt2tokfc5r   nginx-app.1   nginx:latest   ip-172-31-19-242    Running         Running 9 minutes ago
1v41lurx76mh   nginx-app.2   nginx:latest   ip-172-31-19-242    Running         Running 9 minutes ago
yxmeybkccfvj   nginx-app.3   nginx:latest   ip-172-31-19-242    Running         Running 12 seconds ago
yj59kso0yhwu   nginx-app.4   nginx:latest   ip-172-31-19-242    Running         Running 12 seconds ago
p6teq56rr1iy   nginx-app.5   nginx:latest   ip-172-31-19-242    Running         Running 12 seconds ago
(base) labuser@ip-172-31-19-242:~$ docker container ls
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS     NAMES
d8b362b467c3   nginx:latest   "/docker-entrypoint.â€¦"   18 seconds ago   Up 17 seconds   80/tcp    nginx-app.3.yxmeybkccfvj6kued28vgmcc
0bb57496f460   nginx:latest   "/docker-entrypoint.â€¦"   18 seconds ago   Up 17 seconds   80/tcp    nginx-app.5.p6teq56rr1iyugb23gmu3d2m
0c56be2295f7   nginx:latest   "/docker-entrypoint.â€¦"   18 seconds ago   Up 17 seconds   80/tcp    nginx-app.4.yj59kso0yhwuxckk8epqi86j
accf02103adc   nginx:latest   "/docker-entrypoint.â€¦"   10 minutes ago   Up 10 minutes   80/tcp    nginx-app.1.ulyt2tokfc5rudxcioq1b57b
4f6ea528c7b9   nginx:latest   "/docker-entrypoint.â€¦"   10 minutes ago   Up 10 minutes   80/tcp    nginx-app.2.1v41lurx76mh3o1jdcyoe906
(base) labuser@ip-172-31-19-242:~$
```

6. If any container is crashed/stopped/killed due to any reason, docker swarm will launch a replacement container by itself ensuring fault tolerance and availability.

```
[saurabh@ip-172-31-23-224 ~]$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS     NAMES
27975f870de5   nginx:latest   "/docker-entrypoint.â€¦"   4 minutes ago    Up 4 minutes    80/tcp    nginx-app.1.njcmy3n8qe864qhxk0moeoau3
12dfd9425033   nginx:latest   "/docker-entrypoint.â€¦"   4 minutes ago    Up 4 minutes    80/tcp    nginx-app.2.p3p2rm3yj0hjkhkumalonoshq
0ebcafb8e46c   ubuntu         "/bin/bash"              43 hours ago     Up 43 hours               affectionate_rubin
[saurabh@ip-172-31-23-224 ~]$ docker stop 27975f870de5
27975f870de5
[saurabh@ip-172-31-23-224 ~]$
[saurabh@ip-172-31-23-224 ~]$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS     NAMES
12dfd9425033   nginx:latest   "/docker-entrypoint.â€¦"   4 minutes ago    Up 4 minutes    80/tcp    nginx-app.2.p3p2rm3yj0hjkhkumalonoshq
0ebcafb8e46c   ubuntu         "/bin/bash"              43 hours ago     Up 43 hours               affectionate_rubin
[saurabh@ip-172-31-23-224 ~]$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS                  PORTS     NAMES
2d1c92368283   nginx:latest   "/docker-entrypoint.â€¦"   6 seconds ago    Up Less than a second   80/tcp    nginx-app.1.bxk6ntu80sg1d44shakk3dx9p
12dfd9425033   nginx:latest   "/docker-entrypoint.â€¦"   4 minutes ago    Up 4 minutes            80/tcp    nginx-app.2.p3p2rm3yj0hjkhkumalonoshq
0ebcafb8e46c   ubuntu         "/bin/bash"              43 hours ago     Up 43 hours                       affectionate_rubin
[saurabh@ip-172-31-23-224 ~]$
```

We can check the browser port 8081 to confirm if nginx service is running properly.

7. By Default when service is deployed, its attached to INGRESS overlay network for external access not for inter service communication. We would need a custom network which needs to be attached to both/all containers which are expected to communicate with each other. Let's create an overlay network first and then attach it with already existing nginx-app service and then launch a new service.

`docker network create --driver overlay internal-network` # Create a custom network named internal-network

```
(base) labuser@ip-172-31-19-242:~$ docker network create --driver overlay internal-network

date
k2va32ge1sgddcdiz9t8enqvd
Mon Aug  4 13:56:26 UTC 2025
(base) labuser@ip-172-31-19-242:~$
```

`docker service update --network-add internal-network nginx-app` # update nginx-app to use newly created network.

```
(base) labuser@ip-172-31-19-242:~$ docker service update --network-add internal-network nginx-app

nginx-app                                                    network which we created manually        Service to which we are attaching the internal-network
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service nginx-app converged
(base) labuser@ip-172-31-19-242:~$
(base) labuser@ip-172-31-19-242:~$
```

8. `docker service inspect nginx-app --pretty` # Inspect the service to show detailed information in readable format. Make sure it has custom network in the Resources: section.

```
(base) labuser@ip-172-31-19-242:~$ docker service inspect nginx-app --pretty

ID:             fg4q63vk8zm60dzjrdygoe85v
Name:           nginx-app
Service Mode:   Replicated
 Replicas:      1
UpdateStatus:
 State:         completed
 Started:       7 minutes ago
 Completed:     7 minutes ago
 Message:       update completed
Placement:
UpdateConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Update order:      stop-first
RollbackConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Rollback order:    stop-first
ContainerSpec:
 Image:         nginx:latest@sha256:84ec966e61a8c7846f509da7eb081c55c1d56817448728924a87ab32f12a72fb
 Init:          false
Resources:
Networks: internal-network
Endpoint Mode:  vip
Ports:
 PublishedPort = 8081
  Protocol = tcp
  TargetPort = 80
  PublishMode = ingress

(base) labuser@ip-172-31-19-242:~$
```

9. Create a new service from where we will try to communicate with nginx-app service to test the connectivity : `docker service create --name ubuntu --network internal-network ubuntu:latest sleep 100000`

```
(base) labuser@ip-172-31-19-242:~$ docker service create --name ubuntu --network internal-network ubuntu:latest sleep 100000

vjq5724yctk6p8wwdyoi13f1l
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service vjq5724yctk6p8wwdyoi13f1l converged
(base) labuser@ip-172-31-19-242:~$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS          PORTS     NAMES
2ac9e56f4084   ubuntu:latest   "sleep 100000"           19 seconds ago   Up 19 seconds             ubuntu.1.zel1mrgrcsz05a77oao02vw87
90c27283cc2e   nginx:latest    "/docker-entrypoint.â€¦"   13 minutes ago   Up 13 minutes   80/tcp    nginx-app.1.tboatyg5eju0r4rnzq74wp2y1
(base) labuser@ip-172-31-19-242:~$
```

10. Login into one of the container: `docker exec -it <<CONTAINERID>> bash`

Once inside container, install the curl package :

`apt update && apt install curl -y` # Update ubuntu libraries inside container and after that install curl package.

`curl --version` : check if curl installed or not.

```
root@2ac9e56f4084:/# curl --version
curl 8.5.0 (aarch64-unknown-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.6
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp s
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO
root@2ac9e56f4084:/# date
Mon Aug  4 14:27:17 UTC 2025
root@2ac9e56f4084:/#
```

11. Now we can test the connectivity to nginx service using DNS name via : `curl http://nginx-app`

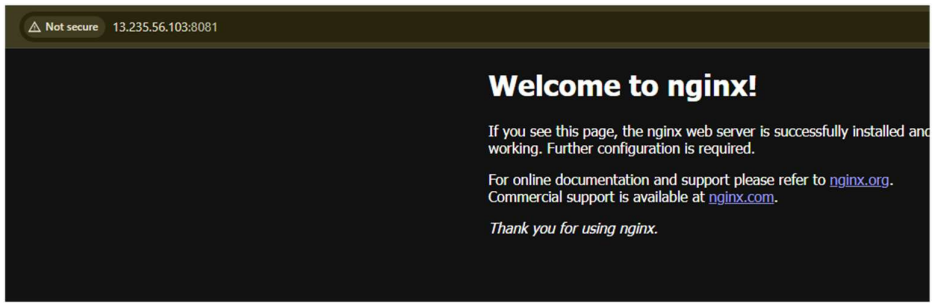
Here we are communicating between container-to-container or service-to-service not externally via a browser, that's why we are using port 80 rather than 8081 (which we will use when we are accessing nginx service from outside).

```
root@2ac9e56f4084:/# curl http://nginx-app
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

12. Check external connectivity to nginx server :



13. Time to clean up the services : `docker service rm <<SERVICE1-ID/NAME>> <<SERVICE2-ID/NAME>>`

```
(base) labuser@ip-172-31-9-72:~$ docker service ls
ID             NAME         MODE         REPLICAS   IMAGE          PORTS
uqsj1cpj8n32   nginx-app    replicated   2/2        nginx:latest   *:8081->80/tcp
jhfb0fnhpbxs   ubuntu       replicated   1/1        ubuntu:latest
(base) labuser@ip-172-31-9-72:~$ docker service rm uqsj1cpj8n32 jhfb0fnhpbxs
uqsj1cpj8n32
jhfb0fnhpbxs
(base) labuser@ip-172-31-9-72:~$ docker service ls
ID        NAME      MODE      REPLICAS   IMAGE     PORTS
(base) labuser@ip-172-31-9-72:~$ docker ps
CONTAINER ID   IMAGE           COMMAND          CREATED          STATUS          PORTS     NAMES
```

14. Delete the network if not needed anymore : `docker network rm <<NETWORK-NAME/ID>>`

```
(base) labuser@ip-172-31-9-72:~$ docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
55c526fd0c7a   bridge            bridge    local
6a0bf8f4cff9   docker_gwbridge   bridge    local
ec734391f431   host              host      local
2wd7b0mvd4q2   ingress           overlay   swarm
7xh5vnslty3k   internal-network   overlay   swarm
4b98f92ed8ca   none              null      local
(base) labuser@ip-172-31-9-72:~$ docker network rm 7xh5vnslty3k
7xh5vnslty3k
(base) labuser@ip-172-31-9-72:~$ docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
55c526fd0c7a   bridge            bridge    local
6a0bf8f4cff9   docker_gwbridge   bridge    local
ec734391f431   host              host      local
2wd7b0mvd4q2   ingress           overlay   swarm
4b98f92ed8ca   none              null      local
(base) labuser@ip-172-31-9-72:~$
```

Docker Stack : Meant for **production** on a **Swarm cluster** with auto-scaling, rolling updates, etc. It's used for deploying and managing multi-container applications on a Docker Swarm cluster instead of a local host (like docker-compose).It uses the same docker-compose.yml format (version 3+). It focuses on production and distributed environments. It requires Swarm mode to be enabled (docker swarm init).

docker swarm init # only required once on manager

Create below docker-compose.yml file :

```
version: '3.8'
services:
  frontend:
    image: nginx
    ports:
      - "8081:80"
    deploy:
      replicas: 3
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: example
```

docker stack deploy -c docker-compose.yml mystack # Create a stack named mystack as per docker-compose.

```
(base) labuser@ip-172-31-20-215:~$ docker stack deploy -c docker-compose.yml mystack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network mystack_default
Creating service mystack_frontend
Creating service mystack_db
```

docker stack ls # List services deployed via stack (services mentioned in docker-compose file)

```
(base) labuser@ip-172-31-20-215:~$ docker stack ls
NAME      SERVICES
mystack   2
```

docker stack services mystack # List services and container status

```
(base) labuser@ip-172-31-20-215:~$ docker stack services mystack
ID             NAME               MODE         REPLICAS   IMAGE             PORTS
ifiwxgnxb1qb   mystack_db         replicated   1/1        postgres:latest
5kspn5bo735b   mystack_frontend   replicated   3/3        nginx:latest      *:8081->80/tcp
(base) labuser@ip-172-31-20-215:~$
```

docker service scale mystack_web=5 # To scale the service "Web" in stack "mystack". Will be reset when stack deploys.

docker stack ps mystack : Display containers launched by services via stack

```
(base) labuser@ip-172-31-20-215:~$ docker stack ps mystack

ID             NAME                 IMAGE             NODE              DESIRED STATE   CURRENT STATE            ERROR
j4rxjv8tl43e   mystack_db.1         postgres:latest   ip-172-31-20-215  Running         Running 2 minutes ago
loa0x8nwuf3x   mystack_frontend.1   nginx:latest      ip-172-31-20-215  Running         Running 2 minutes ago
onruywtzk5y8   mystack_frontend.2   nginx:latest      ip-172-31-20-215  Running         Running 2 minutes ago
x00m8z1mv6qf   mystack_frontend.3   nginx:latest      ip-172-31-20-215  Running         Running 2 minutes ago
(base) labuser@ip-172-31-20-215:~$
```

docker container ls --filter label=com.docker.stack.namespace=mystack # Filter containers from stack name.

```
(base) labuser@ip-172-31-20-215:~$ docker container ls --filter label=com.docker.stack.namespace=mystack
CONTAINER ID   IMAGE            COMMAND                  CREATED         STATUS         PORTS      NAMES
65f6f11377fd   postgres:latest  "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes   5432/tcp   mystack_db.1.j4rxjv8tl43eka4z
f54dee932cba   nginx:latest     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   80/tcp     mystack_frontend.3.x00m8z1mv9
d439362135ff   nginx:latest     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   80/tcp     mystack_frontend.1.loa0x8nwul
f37fc44db5bf   nginx:latest     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   80/tcp     mystack_frontend.2.onruywtzkc
(base) labuser@ip-172-31-20-215:~$
```

docker container stats # All container metrics

```
CONTAINER ID   NAME                                     CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O
65f6f11377fd   mystack_db.1.j4rxjv8tl43eka4qhg8tfey2z   0.00%     24.33MiB / 15.4GiB    0.15%     3.4kB / 0B    0B / 41.8MB
f54dee932cba   mystack_frontend.3.x00m8z1mv6qf5t4vfh0rdekn9   0.00%     4.363MiB / 15.4GiB    0.03%     3.4kB / 0B    0B / 12.3kB
d439362135ff   mystack_frontend.1.loa0x8nwuf3xfugdnciils82l   0.00%     4.344MiB / 15.4GiB    0.03%     3.4kB / 0B    0B / 12.3kB
f37fc44db5bf   mystack_frontend.2.onruywtzk5y81sjszzi5821rc   0.00%     4.363MiB / 15.4GiB    0.03%     3.4kB / 0B    0B / 12.3kB
```

docker stack rm mystack # Removes all resources inside stack.

```
(base) labuser@ip-172-31-20-215:~$ docker stack rm mystack
Removing service mystack_db
Removing service mystack_frontend
Removing network mystack_default
(base) labuser@ip-172-31-20-215:~$
```

Feature	Docker Compose	Docker Stack
Deployment target	Single host	Swarm cluster (multi-node)
Orchestration support	No built-in orchestration	Uses Swarm as orchestrator
Command	docker-compose	docker stack
Service scaling/failure	Manual	Swarm automatically replaces failed replicas
Rolling updates	Not supported	Supported in deploy section
Health checks, placement	Not supported	Supported via deploy options
Volumes/networks	Local only	Cluster-scoped